```
    public DeleteModel(IPropertyService propertyService)      ❶
    {                                                         ❶
        _propertyService = propertyService;                   ❶
    }                                                         ❶

    public Property Property { get; set; }                    ❷

    [BindProperty(SupportsGet = true)]                        ❸
    public int Id { get; set; }                               ❸

    public async Task<IActionResult> OnGetAsync()             ❹
    {                                                         ❹
        Property = await _propertyService.FindAsync(Id);      ❹

        if (Property == null)                                 ❹
        {                                                     ❹
            return NotFound();                                ❹
        }                                                     ❹
        return Page();                                        ❹
    }                                                         ❹

    public async Task<IActionResult> OnPostAsync()            ❺
    {                                                         ❺
        await _propertyService.DeleteAsync(Id);               ❺
        return RedirectToPage("./Index");                     ❺
    }                                                         ❺
}
```

❶ The injected context is replaced with the IPropertyService.

❷ Remove the unnecessary BindProperty attribute from the Property property.

❸ Add a bound property for the key value, replacing the handler parameters.

❹ Remove the null check against the key value because you will use a route constraint instead.

❺ Use the service to “delete” the entity.

Just as with the Edit page, you only want the Delete page to be reached if an integer value is included in the URL, so you turn to the Delete.cshtml and add a route constraint as part of a route template:

```
@page "{id:int}"
```